

Tipos Primitivos y Objetos

Como ya sabemos en Javascript hay **6 tipos primitivos** de datos: **string, number, boolean, null, undefined y symbol**. También sabemos que un tipo primitivo **almacena un valor y no una referencia**. Pero, por otro lado, estaría bien poder ejecutar algunos métodos sobre dichos objetos pero manteniendo esos objetos ligeros.

Para conseguir esto, en JS se emplean los **objetos envolventes**. Es decir, cuando se usa la notación `.` (o corchetes) sobre un valor primitivo, en JavaScript se **crea un objeto** con la información del mismo, se **ejecuta el método** y, al final, se **destruye** dejando el valor primitivo como estaba.

Existen tres de estos tipos envolventes: `String`, `Number` y `Boolean`.

```
let str = "Hola";
```

```
str.toUpperCase()
```

```
>"HOLA"
```

```
let num = 12.34
```

```
num.toFixed(1)
```

```
>"12.3"
```

```
let b = true;
```

```
b.valueOf()
```

```
>true
```

```
12.toFixed(2)
```

```
> Uncaught SyntaxError: Invalid or unexpected token
```

```
12..toFixed(2)
```

```
"12.00"
```

Como se puede ver, en el caso de los números enteros hay que emplear **dos puntos** para evitar que uno solo se confunda con el separador decimal.

NOTA: Aunque es posible no es conveniente emplear el operador **new** para crear objetos de las clases envolventes, dado que tiene consecuencias no deseadas:

```
let num = new Number(0);

typeof num

>object

num?"true":"false"

>true
```

Como ya sabemos, emplear el constructor **sin new** es perfectamente válido (en este caso **retorna un dato primitivo**)

```
num = Number(0);

typeof num

>"number"

(num)?"true":"false"

// devuelve false porque el num seria 0 y el 0 lo toma como false

>"false"
```

String

Un objeto `String` permite representar una cadena. Cualquier conjunto de caracteres entre **comillas dobles o simples** se considera una cadena y también se puede crear una cadena con la función `String` (sin `new`):

Propiedades

- `length` : Retorna la longitud de una cadena en caracteres. Intentar usarlo sobre variables `undefined` o `null`, dará un **error de ejecución**

Algunos métodos

- `charAt(pos)` : Retorna el carácter de la cadena que se encuentra en la posición **pos** (empezando a contar desde cero). El uso de posiciones negativas o fuera de los límites de la cadena, retorna una cadena vacía.
- `charCodeAt(pos)` : Retorna el código UNICODE del carácter de la cadena que se encuentra en la posición `pos` (empezando a contar desde cero). El uso de posiciones negativas o fuera de los límites de la cadena, retorna **NaN**.
- `concat(listaCadenas)` : Permite concatenar varias cadenas
- `endsWith(cadena)` : Comprueba si el objeto cadena finaliza en la cadena pasada como argumento
- `fromCharCode(listacódigos)` : Método estático que retorna una cadena a partir de una serie de valores UNICODE.
- `includes(cadena)` : Comprueba si el objeto cadena incluye la cadena pasada como argumento
- `indexOf(cadena[, pos])` : Retorna la posición de la cadena indicada en el objeto cadena sobre el que se ejecuta. La búsqueda se inicia en **pos** (0 si no se especifica nada). Si la subcadena no está, retorna el un **-1**
- `lastIndexOf(cadena[, pos])` : Similar a la anterior, sólo que comienza la búsqueda **desde el final de la cadena**.
- `localeCompare(cadena)` : Compara la cadena objeto con la pasada teniendo en cuenta el idioma del equipo
- `match(exprReg)` : Retorna todas las partes de la cadena que cumplen con la expresión regular como un **array de Strings**. NOTA: ver las **expresiones regulares** más adelante en esta unidad
- `repeat(num)` : Retorna la cadena repetida **num** veces
- `replace(cad1, cad2)` : Sustituye todas las apariciones de **cad1** por **cad2**, donde **cad1** puede ser una **expresión regular** (en este ultimo caso hay un tercer parámetro que puede ser **\$\$** (se sustituye por **\$**), **\$&** inserta la cadena encontrada, **\$`** inserta la porción **delante** de la encontrada, **\$'** inserta la parte **detrás** de la encontrada, **\$n** inserta la aparición **n** de la cadena encontrada
- `search(exprReg)` : Retorna la posición de la primera aparición de una subcadena que cumpla con la expresión regular. Si no existe, retorna un **-1**

- `slice(posIni,posFin)` : Retorna una cadena formada por todos los caracteres tomados desde la posición **pos1** hasta la posición **pos2** de la cadena sobre la que se ejecuta. Si no se especifica la posición final, retorna todos los que haya hasta el final de la cadena
- `split(sep[,limite])` : Retorna un **array** con todas las subcadenas de la cadena sobre la que se ejecuta, empleando **sep** como carácter delimitador para la separación. Si se especifica **limite**, se retornan tantas subcadenas como las indicadas en este parámetro
- `startsWith(cadena)` : Comprueba si el objeto cadena empieza con la cadena pasada como argumento
- `substring(pos,car)` : Retorna una cadena formada por **car** caracteres tomados a partir de la posición **pos** de la cadena sobre la que se ejecuta. Si no se especifica el número de caracteres, retorna todos los que haya hasta el final de la cadena
- `toLowerCase()` / `toUpperCase()` : Retorna la cadena en minúsculas / mayúsculas
- `toLocaleLowerCase()` / `toLocaleUpperCase()` : Retorna la cadena en minúsculas / mayúsculas teniendo en cuenta el idioma del navegador.
- `toString()` : Retorna la cadena
- `trim()` : Retorna la cadena sin espacios al principio y al final
- `trimEnd()` / `trimStart()` : Retorna la cadena sin espacios al final / principio de la misma
- `valueOf()` : retorna la cadena.

Además de estos métodos, la clase `String` dispone de una serie de métodos que permiten **encapsular funcionalidades HTML** que **ACTUALMENTE ESTÁN EN DESUSO** (deprecated)

NOTA: Las cadenas son **inmutables** (no cambian el valor) y se puede acceder a sus elementos como si fuera una matriz (no es estándar ECMAScript). Se suele utilizar más el método `charAt`:

```
str = "Hola";
```

```
"Hola"
```

```
str[1]
```

```
"o"
```

```
str[1] = "0"
```

```
"0"
```

```
str
```

```
"Hola"
```

```
str = "Hola"
```

```
str.charAt(2)
```

```
>"l"
```

NOTA: Han sacado un método nuevo que es `at(posición)`. Si se le pasa un número negativo devuelve empezando por el final (en este caso, empieza a contar desde 1 en vez de desde 0)

```
str.at(-2)
```

```
>'l'
```

```
str.at(1)
```

```
>'o'
```

```
str.at(-1)
```

```
>'a'
```

Number

Propiedades

- `MAX_VALUE` : máximo valor posible
- `MIN_VALUE` : mínimo valor posible
- `MIN_SAFE_INTEGER` : mínimo entero que se puede usar sin obtener resultados impredecibles ($-(2^{53}-1)$)
- `MAX_SAFE_INTEGER` : máximo entero posible ($2^{53}-1$)
- `NEGATIVE_INFINITY` : representa el infinito negativo
- `NaN` : representa que el dato no es un número
- `POSITIVE_INFINITY` : representa el infinito positivo

```
Number.MAX_SAFE_INTEGER
```

```
9007199254740991
```

```
Number.MAX_SAFE_INTEGER+1
```

```
9007199254740992
```

```
Number.MAX_SAFE_INTEGER+2
```

```
9007199254740992
```

Number.MAX_SAFE_INTEGER+4

9007199254740996

Number.MAX_SAFE_INTEGER+3

9007199254740994

Métodos

- `isNaN()` / `isFinite()` / `isInteger()` / `isSafeInteger()` : Evalúa el valor del número
- `parseFloat(valor)` / `parseInt(valor)` : Retorna el valor como un decimal o un entero
- `toExponential([valor])` : Convierte un número a su notación exponencial con tantos decimales como se indique
- `toFixed([decimales])` : Retorna una cadena con el número con el número de **decimales** especificados (redondeando su valor)
- `toPrecision(n)` = n cifras significativas desde la primera cifra que no sea cero. Los ceros que están antes de la primera cifra distinta de cero no cuentan.

Así que piensa:

0.00340000 → ignoro los ceros iniciales → 3 4 0 0 0 0 → 6 cifras significativas.

Y una diferencia MUY importante:

`toFixed(3)` → 3 cifras después de la coma

`toPrecision(3)` → 3 cifras significativas en total

- `toString([base])` : Retorna el número como una cadena (en la base especificada)
- `valueOf()` : Retorna el valor del número

`1234.567.toExponential(7)`

`"1.2345670e+3"`

`1234.567.toExponential(2)`

`"1.23e+3"`

`12.34567.toFixed(6)`

`"12.345670"`

`12.34567.toFixed(2)`

`"12.35"`

`12.34567.toFixed(0)`

```
12.34.toPrecision(6)
```

```
>"12.3400"
```

```
12.34.toPrecision(3)
```

```
>"12.3"
```

```
12.34.toPrecision(2)
```

```
>"12"
```

```
0.34.toPrecision(6)
```

```
>"0.340000"
```

```
0.0034.toPrecision(6)
```

```
>"0.00340000"
```

```
12345..toPrecision(2)
```

```
>"1.2e+4"
```

```
1.2e+4.toPrecision(4)
```

```
>"1.200e+4"
```

```
1024..toString()
```

```
"1024"
```

```
1024..toString(16)
```

```
"400"
```

```
1024..toString(2)
```

```
"100000000000"
```

```
1024..toString(23)
```

```
"1lc"
```

Boolean

El objeto **boolean** permite almacenar valores lógicos. Sólo dispone de los métodos

- `toString()` : Retorna el valor booleano como una cadena
- `valueOf()` : Retorna el valor del objeto booleano

```
true.toString()
```

```
>'true'
```

Date

El objeto **Date** nos permite trabajar con **instantes de tiempo**. Dispone de **cuatro constructores** :

- `Date()` : Constructor sin argumentos. Retorna el **instante actual**
- `Date(milisegundos)` : Construye un objeto **Date** que representa el instante especificado en **milisegundos**.
- `Date(cadena)` : Retorna una fecha que apunta al instante indicado en **cadena**. El estándar especifica el siguiente formato : **YYYY-MM-DDTHH:mm:ss.sssZ** donde **T** indica el comienzo de la parte horaria y **Z** se especifica cuando se usa el **tiempo universal coordinado** (que sustituya a **GMT**). La **Z** se puede sustituir por la expresión **+/-HH:MM** para especificar el desfase horario (si se **omite**, se emplea el desfase horario en función de la configuración del equipo, lo que es la mejor opción). Se puede omitir la parte de la fecha y/o la hora y/o la zona horaria.

También se suelen aceptar cadenas con el formato **NombreMes día, año [hh:mm:ss]** o **YYYY/MM/DD [HH:MM]** aunque no sean valores estándar (dado que no son estándar la hora se asume en formato local)

- `Date(año,mes,día[,hora,minuto,segundo[)])` : Retorna una fecha en base a los datos especificados. Como en JAVA el mes **empieza a contar desde 0 (Enero)**

```
// A partir del 1/01/1970 te suma los milisegundos(Timestamp) que se han indicado,  
construyendo la fecha correspondiente. 86400000(24*60*60*1000)
```

```
document.write(new Date(86400000)+" --> 1 día en milisegundos<br/>");
```

Lo mejor es utilizar timestamp, (para guardar o extraer fechas de la BD). Esta forma es de las

más habituales en la gestión de fechas, con el fin de que funcione en cualquier parte del mundo.

Una vez tenemos el timestamp, podemos guardarlo por ejemplo en BD, y recuperarlo posteriormente para construir una fecha.

```

let fecha = new Date()
> undefined
fecha
> Sun Sep 19 2021 12:01:00 GMT+0200 (hora de verano de Europa central)

// con getTime() cogemos el número de milisegundos que han pasado desde el 01/01/1970

fecha.getTime()
> 1632045660996
let fecha2 = new Date(1632045660996)
> undefined
fecha2
> Sun Sep 19 2021 12:01:00 GMT+0200 (hora de verano de Europa central)

```

Métodos

- `getDate()` : retorna el día del mes (1-31)
- `getDay()` : retorna el día de la semana (0-Domingo a 6-Sábado)
- `getFullYear()` : retorna el año con **cuatro dígitos**
- `getHours()` : retorna la hora (0-23)
- `getMilliseconds()` : retorna los milisegundos de la fecha (0-999)
- `getMinutes()` : retorna los minutos (0-59)
- `getMonth()` : retorna el número del mes (0-11)
- `getSeconds()` : retorna los segundos (0-59)
- `getTime()` : retorna la fecha en milisegundos desde el **1 de enero de 1970**
- `getTimezoneOffset()` : retorna el **desfase** respecto a la **UTC** en **minutos**
- `getUTCDate()`, `getUTCDay()`, `getUTCFullYear()`, `getUTCHours()`, `getUTCMilliseconds()`, `getUTCMinutes()`, `getUTCmonth()`, `getUTCSeconds()` : retorna los periodos indicados en hora UTC (dos o una hora menos en función de la época)
- `parse(cadena)` : método **estático** que retorna el **número de milisegundos** que corresponde a la cadena recibida. Ver los posibles formatos en el **constructor** de la clase. Si la fecha es errónea, retorna **NaN**
- `setDate(valor)`, `setDay(valor)`, `setFullYear(valor)`, `setHours(valor)`, `setMilliseconds(valor)`, `setMinutes(valor)`, `setMonth(valor)`, `getSeconds(valor)` :

permiten modificar partes de una fecha

- `setUTCDate(valor)`, `setUTCDay(valor)`, `setUTCFullYear(valor)`, `setUTCHours(valor)`, `setUTCMilliseconds(valor)`, `setUTCMinutes(valor)`, `setUTCmonth(valor)`, `getUTCSeconds(valor)` : similar al anterior pero considerando el desfase horario
- `toDateString()`, `toISOString()`, `toLocaleDateString()`, `toLocaleTimeString()`, `toTimeString()`, `toUTCString()`, `toString()` : retorna la fecha como una cadena (`toGMTString()` está en desuso)

<code>toDateString()</code>	Mon, 06 Aug 2012
<code>toLocaleDateString()</code>	lunes, 06 de agosto de 2012
<code>toLocaleString()</code>	lunes, 06 de agosto de 2012 13:09:41
<code>toLocaleTimeString()</code>	13:09:41
<code>toUTCString()</code>	Mon, 06 Aug 2012 11:09:41 GMT
<code>toISOString()</code>	2012-08-06T11:09:41.027Z
<code>toString()</code>	Mon Aug 06 2012 13:09:41 GMT+0200

- `UTC()` : retorna el número de milisegundos considerando la fecha en hora UTC
- `valueOf()` : retorna la fecha como milisegundos

Ejemplos:

```
fecha = new Date()  
> Fri Sep 24 2021 08:17:59 GMT+0200 (hora de verano de Europa central)  
fecha.toDateString()  
> 'Fri Sep 24 2021'  
fecha.toLocaleDateString()  
> '24/9/2021'
```

```
fecha.getDate()  
> 24  
fecha.getFullYear()  
>2021  
fecha.getHours()  
> 8
```

Math

El objeto **Math** permite ejecutar diferentes métodos matemáticos.

Propiedades

- **E** : Representa el número e.
- **LN2** : Retorna el logaritmo neperiano de 2
- **LN10** : Retorna el logaritmo neperiano de 10
- **LOG2E** : Retorna el logaritmo en base 2 de e
- **LOG10E** : Retorna el logaritmo en base 10 de e
- **PI** : Retorna el valor de π
- **SQRT1_2** : Retorna la raíz cuadrada de $\frac{1}{2}$
- **SQRT2** : Retorna la raíz cuadrada de 2

Métodos

- **abs(x)** : Retorna el valor absoluto
- **acos(x)**, **asin(x)**, **atan(x)**, **atan2(x)** : Retorna el arcoseno, arcocoseno, arcotangente y arcocotangente de x
- **ceil(x)** : Retorna el primer entero **mayor** que x
- **cos(x)**, **sin(x)**, **tan(x)**, **tan2(x)** : Retorna el seno, coseno, tangente, cotangente de x
- **exp(x)** : Retorna e^x
- **floor(x)** : Retorna el primer entero **menor** que x
- **log(x)** : Retorna el logaritmo neperiano de x
- **max(lista_valores)**, **min(lista_valores)** : Retorna el mayor/menor valor de la lista
- **pow(x,y)** : Retorna x^y
- **random()** : Retorna un número aleatorio entre 0 y 1 (sin incluir 1)

- `round(x)` : Retorna el entero más cercano a x
- `sqrt(x)` : Retorna la raíz cuadrada de x
- `trunc(x)` : Retorna la parte entera de x

Por ejemplo:

```
Math.ceil(5.6)
```

6

```
Math.floor(5.6)
```

5